CHORALE · MODEL EVALUATION

Chorale Coder — Model Evaluation Report

A head-to-head evaluation of **11 language models** as the coding engine behind Chorale's `coder` agent — measured on correctness, speed, and true per-task cost, through the production salvage-and-verify pipeline.

L1 → L10 algorithmic ladder | 11 models · 4 providers | Behavioral grading (hidden tests) | Normalized per-token cost

EXECUTIVE SUMMARY

- **Seven of eleven models scored a perfect 10/10.** Correctness *saturates* on these tasks — the decision comes down to **speed and cost**, where the field spreads by more than an order of magnitude.
- **Value winner & default: Gemma-4-31B (Hugging Face)** — 9/10 at **≈$0.00** and **13 seconds** total, one-shotting almost everything. The nearest 10/10 model is 7–10× slower.
- **Best escalation: gpt-oss-120B (Fireworks)** — the **cheapest 10/10** at **~$0.013** for the whole ramp, and among the fastest. Now wired in as the heavy-tier fallback behind Gemma.
- **The premium models earned nothing here.** GLM-5.2 and Kimi-K2.6 cost **~18×** more than gpt-oss-120B for the *same* score; DeepSeek-V4-Pro and MiniMax-M3 ~10×.

11	10	7	~60×	53×
MODELS TESTED	DIFFICULTY LEVELS	PERFECT 10/10	COST SPREAD, SAME RESULT	SPEED SPREAD

1 Context & Goal

Chorale is a model-agnostic, multi-agent system: any agent can point at any model — local (Ollama) or serverless (Hugging Face, Fireworks, Anthropic) — with no code change. The `coder` agent is the flagship, and the design goal is explicit: **bring the best out of any model, from the smallest local model to the largest frontier model, and route work to the best-value option.**

This evaluation answers a concrete, practical question: *among the models we can actually reach, which should the coder default to, and which is worth escalating to when the default falls short?* The honest answer required running the same workload through every candidate and measuring what it truly costs and delivers — not reading a billing dashboard, which proved misleading.

2 What Was Tested — the L1 → L10 Ladder

Each level asks the coder to produce a single, self-contained ES-module (`solution.mjs`) exporting one function or class. Difficulty escalates:

LEVEL	TASK	WHAT IT STRESSES
L1	Roman numeral conversion	Basic mapping & loops
L2	Balanced-bracket validator	Stack logic, edge cases (`([)]`)
L3	Arithmetic expression evaluator	Operator precedence, parsing
L4	LRU cache (class)	Stateful structure, eviction order
L5	JSON parser (no `JSON.parse`)	Recursive descent, string escapes
L6	Regex matcher (`.` and `*`)	Recursion / DP, backtracking
L7	Topological sort	Graph traversal, cycle detection
L8	CSV parser (quotes, escapes, newlines)	Stateful text parsing
L9	Mini-interpreter (variables + arithmetic)	State across statements
L10	Dijkstra shortest path	Weighted graph + priority queue

Grading is behavioral, not textual. Each solution is imported and executed against hidden test cases. The grader resolves the export leniently (exact name → `default` → the sole callable export), so a cosmetic name mismatch never masks correct logic. A level passes only if every assertion holds.

3 Methodology

The pipeline is the real one. Models run through Chorale's production coder agent, including content-level **tool-call salvage** (recovering file writes that weak models emit as text or backticked blocks), the **syntax verify-and-repair loop** (esbuild checks each file, feeds errors back with escalating temperature), and **auto-export rescue**. The numbers reflect the coder *as shipped*, not a raw API call.

Metrics per level: pass/fail, wall-clock seconds, and input/output tokens summed across all fallback and repair attempts.

Cost model. Raw billing totals are unreliable — they bundle prior-session usage and vary with cache-hit rate. Cost here is **normalized**: recomputed from *this ramp's* measured tokens × each model's Fireworks list rates, charging input at the cached rate (realistic — the ~3k-token system prompt is reused every turn) plus output at the output rate. It is reproducible and cross-checks against billed figures within caching variance. Gemma-4-31B runs on Hugging Face (not on these rate cards); its billing delta showed the ramp at **<$0.01**.

4 Results — Per-Level Pass Grid

MODEL	L1	L2	L3	L4	L5	L6	L7	L8	L9	L10	SCORE
Gemma-4-31B	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	**9/10**
gpt-oss-120B	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	**10/10**
DeepSeek-V4-Flash	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	**10/10**
MiniMax-M2.7	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	**10/10**
Qwen3.7-Plus	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	**10/10**
DeepSeek-V4-Pro	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	**10/10**
MiniMax-M3	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	**10/10**
GLM-5.2	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	**10/10**
Kimi-K2.6	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	**10/10**
gpt-oss-20B	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	**9/10**
Nemotron-3-Ultra (NVFP4)	✓	✓	✗	✗	✓	✓	✓	✓	✗	✓	**7/10**

5 Master Comparison — Score · Speed · Tokens · Cost

Sorted by score, then normalized cost. **Ramp $** = normalized cost for the full L1→L10 run (cached input + output at list rates). **Out $/M** is the output list price — the dominant cost driver.

MODEL	SCORE	TIME	IN TOK	OUT TOK	OUT $/M	RAMP $
Gemma-4-31B DEFAULT	**9/10**	**13s**	31,663	6,036	(HF)	**≈$0.00**
gpt-oss-120B ESCALATION	10/10	130s	127,322	18,649	$0.60	**$0.013**
DeepSeek-V4-Flash	10/10	457s	281,863	25,017	$0.28	$0.016
MiniMax-M2.7	10/10	96s	156,820	17,270	$1.20	$0.030
Qwen3.7-Plus	10/10	122s	n/a	n/a	$1.60	~$0.09
DeepSeek-V4-Pro	10/10	437s	283,613	23,595	$3.48	$0.125
MiniMax-M3	10/10	571s	n/a	n/a	$1.20	~$0.14
GLM-5.2	10/10	417s	343,559	40,927	$4.40	$0.228
Kimi-K2.6	10/10	693s	311,697	49,082	$4.00	$0.246
gpt-oss-20B	9/10	136s	29,769	10,817	$0.30	$0.004
Nemotron-3-Ultra (NVFP4)	7/10	312s	71,541	11,184	$2.40	$0.035

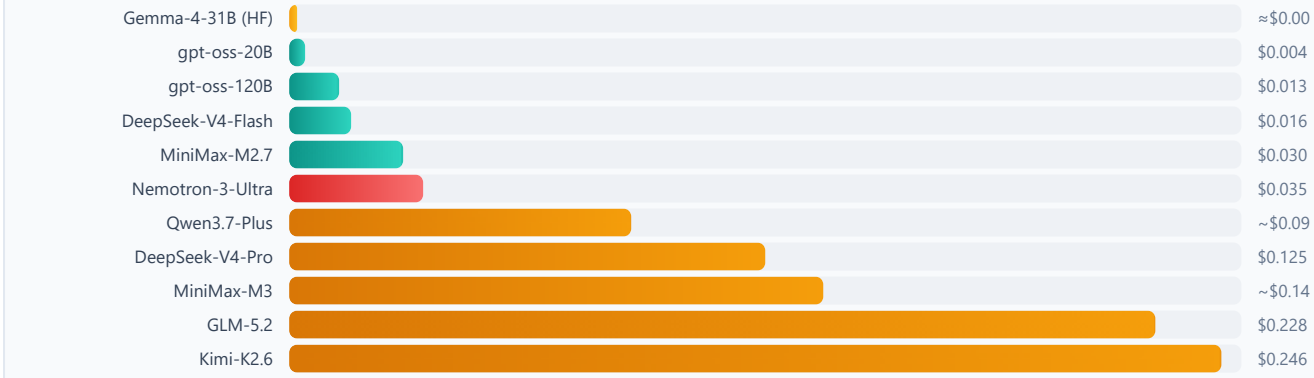
Decontaminated: raw dashboard totals were GLM $0.60 and Kimi **$2.75**, but those bundled earlier-session usage. Recomputed from this ramp's tokens, Kimi's true cost is ~$0.25 — ~18× the cheap tier, not the ~90× the raw total implied. Qwen3.7-Plus / MiniMax-M3 report no token usage via the API; their figures are the billed ramp-only accrued cost.

6 Cost & Speed, Visualized

Default (best value) · Cheap 10/10 · Premium (overpriced here) · Underwhelming

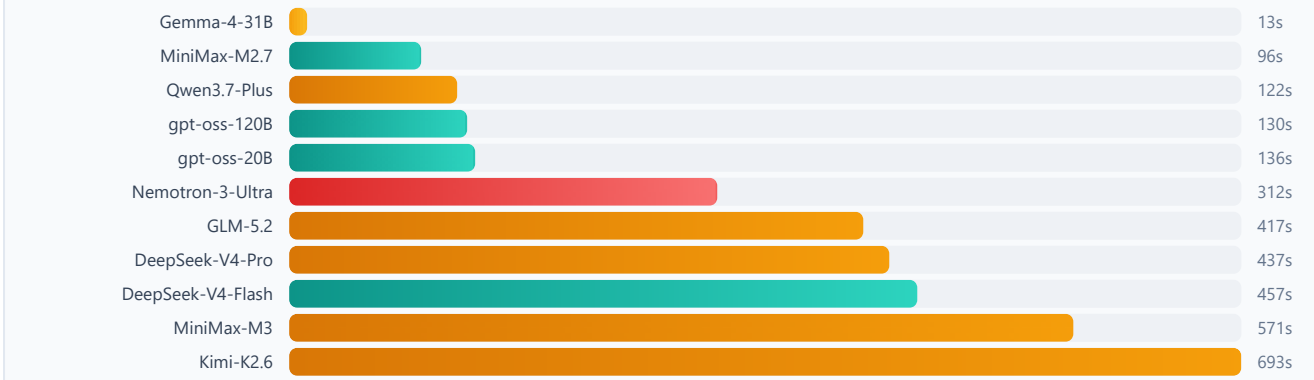
Normalized ramp cost (full L1→L10, USD) — lower is better

~60× spread for identical 10/10 results, driven by output token pricing ($0.28 → $4.40 / M).



Total wall-clock for the ten levels (seconds) — lower is better

53× spread. Fast models one-shot; slow ones burn minutes on internal reasoning that changes no outcome here.



7 Analysis

Correctness saturates — efficiency is the real axis

Seven models scored 10/10; the two 9/10 results each miss one different level. Raw capability is effectively solved for any competent model on this ladder, which throws the decision entirely onto **cost and speed** — where the field spreads by more than an order of magnitude.

Cost is an output-pricing story

Normalized ramp cost ranges **~$0.004 → ~$0.25** — a ~60× spread for identical results. The driver is **output token pricing** ($0.28/M → $4.40/M, a 15.7× range) multiplied by how many tokens each model emits. Reasoning-heavy models lose twice: they charge the most per output token *and* generate the most of them.

The value tiers

TIER	MODELS	VERDICT
Default	Gemma-4-31B	9/10, ≈$0, 13s. In a class of its own on value.
Escalation	gpt-oss-120B · DeepSeek-V4-Flash · MiniMax-M2.7	Perfect scores at trivial cost. gpt-oss-120B is the pick — cheapest and fast.
Premium	DeepSeek-V4-Pro · MiniMax-M3 · GLM-5.2 · Kimi-K2.6	Same 10/10 for 10–18× the cost. No justification on this workload.
Weak	Nemotron-3-Ultra (NVFP4)	7/10 and ~$0.035 — pays more for a worse score. 4-bit quant appears to cost accuracy.

Notable findings

• **Bigger isn't better within a family.** MiniMax-M2.7 and M3 score identically, but M3 is ~6× slower and ~3× pricier for zero gain. • **"Flash" is a misnomer.** DeepSeek-V4-Flash has the cheapest output rate but was one of the *slowest* models (457s) — it reasons heavily. • **The small sleeper:** gpt-oss-20B scored 9/10 at ~$0.004, nearly free, tripping only on L6 regex. • **Gemma's one gap is diagnostic:** the level it misses — L9, a stateful mini-interpreter — is exactly what the escalation model is for.

8 The Decision

Default: Gemma-4-31B → Escalate: gpt-oss-120B

Gemma handles 9/10 of this workload at near-zero cost and interactive speed — the right default for the overwhelming majority of turns. **gpt-oss-120B** sits directly behind it as the heavy-tier fallback: the cheapest perfect scorer, fast and transparent — the right model for the hard cases (like stateful interpreters) where Gemma slips. The premium models are **not** in the default path; they earned no measurable advantage to justify their 10–18× cost. *This architecture is now wired into Chorale's agents and profiles.*

9 Limitations & Honest Caveats

1. **N=1 per level.** Single runs. The 10/10 results are trustworthy given frontier determinism, but the 9-vs-10 boundary could move by one level on a re-run.
2. **Puzzles, not projects.** Single-file algorithmic tasks. They do *not* measure multi-file refactoring, debugging, long-horizon planning, or heavy tool use — the arenas where the biggest frontier models most likely justify their cost. A model that looks "overpriced" here may pull ahead on real engineering; the right next step is a multi-file, real-project benchmark.
3. **Cost is normalized, not billed.** Recomputed from measured tokens × list rates (cached input + output); real bills vary with actual cache-hit rates. GLM-5.2 / Kimi-K2.6 dashboard totals were excluded as prior-session-contaminated; Gemma runs on HF (≈$0 per the billing delta).
4. **Two models report no token usage** (Qwen3.7-Plus, MiniMax-M3) via the Fireworks API; their cost is the billed ramp-only accrued figure.

10 Reproducibility

```
# Single model, open-ended ramp (streams a live log; stops on failure/slowness)
pnpm exec tsx eval/coder-ramp.ts "hf:google/gemma-4-31B-it"

# Head-to-head bake-off across N models, L1..MAX, full pass grid + tokens
pnpm exec tsx eval/coder-bakeoff.ts 10 \
  "hf:google/gemma-4-31B-it" \
  "fireworks:accounts/fireworks/models/gpt-oss-120b" \
  "fireworks:accounts/fireworks/models/minimax-m2p7"
```

Harness: `eval/coder-bakeoff.ts` · Challenges & grader: `eval/challenges.ts` · Condensed board: `eval/RAMP-LEADERBOARD.md`

11 Addendum — Real-Engineering Validation

The ramp measures single-file puzzles. To test that caveat head-on, the three finalists were run through a **multi-file, multi-step engineering** benchmark — build a library, debug a broken codebase, get async concurrency right, ship a stateful CLI — with the coder using its full toolset (including `bash`, so it runs tests and iterates like a developer). Each project is graded with **partial credit** by executing the result; every grader was validated against reference good/bad solutions first.

MODEL	KV STORE	DEBUG	ASYNC POOL	TODO CLI	OVERALL	TIME	COST
Gemma-4-31B	8/8	7/7	5/5	6/6	**26/26 · 100%**	**32s**	**≈$0.00**
gpt-oss-120B	8/8	7/7	4/5	6/6	25/26 · 96%	68s	$0.0071
MiniMax-M2.7	8/8	7/7	5/5	6/6	**26/26 · 100%**	73s	$0.0246

The four projects test distinct real skills — **build** (a KV-store library with TTL + LRU across two files), **debug** (localize and fix three planted bugs in a broken `lib/` until its suite passes), **async correctness** (a bounded-concurrency promise pool: order, cap, parallelism, error propagation, in-flight settling), and **integration** (a todo CLI persisting to JSON across processes).

> **The value king holds up.** Gemma-4-31B scored a perfect **26/26** on real multi-file engineering while being ~2× faster than either frontier model and effectively free. MiniMax-M2.7 also went 100%, but at ~3.5× the cost and ~2.3× the time. gpt-oss-120B slipped to 96% — its promise pool failed to reject when a task threw (a real error-handling gap); everything else was perfect. On this evidence the "frontier models justify their cost on real work" hypothesis does **not** hold at this task scale: Gemma matches or beats them. *(Caveat: N=1, and these are small self-contained projects, not thousand-line codebases.)*

Chorale is an open-source, model-agnostic multi-agent system. This report reflects measurements taken through its production coder pipeline; numbers will shift as models, prices, and the pipeline evolve. Generated from `docs/model-evaluation-report.md`.